- *File I/O*—We've already demonstrated how to use stream processing for file I/O. Although we've focused on line-by-line reading and writing for the examples here, we can also use the library for processing binary files.
- *Message processing, state machines, and actors*—Large systems are often organized as a system of loosely coupled components that communicate via message passing. These systems are often expressed in terms of *actors*, which communicate via explicit message sends and receives. We can express components in these architectures as stream processors. This lets us describe extremely complex state machines and behaviors using a high-level, compositional API.
- *Servers, web applications*—A web application can be thought of as converting a stream of HTTP requests to a stream of HTTP responses.
- *UI programming*—We can view individual UI events such as mouse clicks as streams, and the UI as one large network of stream processors determining how the UI responds to user interaction.
- *Big data, distributed systems*—Stream-processing libraries can be *distributed* and *parallelized* for processing large amounts of data. The key insight here is that the nodes of a stream-processing network need not all live on the same machine.

If you're curious to learn more about these applications (and others), see the chapter notes for additional discussion and links to further reading. The chapter notes and code also discuss some extensions to the `Process` type we discussed here, including the introduction of *nondeterministic choice*, which allows for concurrent evaluation in the execution of a `Process`.

15.5 Summary

We began this book with a simple premise: that we assemble our programs using only pure functions. From this sole premise and its consequences, we were led to explore a completely new approach to programming that's both coherent and principled. In this final chapter, we constructed a library for stream processing and incremental I/O, demonstrating that we can retain the compositional style developed throughout this book even for programs that interact with the outside world. Our story, of how to use FP to architect programs both large and small, is now complete.

FP is a deep subject, and we've only scratched the surface. By now you should have everything you need to continue the journey on your own, to make functional programming a part of your own work. Though good design is always difficult, expressing your code functionally will become effortless over time. As you apply FP to more problems, you'll discover new patterns and more powerful abstractions.

Enjoy the journey, keep learning, and good luck!